

Train Large Models Across the World

Accelerating Training on Geo-Distributed Clusters via Heterogeneity-Aware Collective Communication

Yuxiang Lin Tianjun Yuan

{yl925, ty117}@duke.edu

April 2025

Outline

- 1 Background and Introduction
- 2 Proposed Method
- 3 Training Algorithm Implemented
- 4 Heterogeneous Training Time Analysis
- 5 Evaluation

Outline

- 1 Background and Introduction
- 2 Proposed Method
- 3 Training Algorithm Implemented
- 4 Heterogeneous Training Time Analysis
- 5 Evaluation

Scaling Models vs Scaling Infrastructure

Larger, Deeper Models (e.g. From 405B)		Faster, Smarter Networks	
Metric	Value	Aspect	Trend
Hidden Size	16,000 → 96,000	Network Scope	Intra-area → Inter-city
Transformer Layers	128 → 384	Bandwidth	Increases drastically
Total Parameters	405B → 405T	Latency	Drops significantly
Training Cost	Extremely High		
<i>Challenges: stability, compute, memory, training time</i>		<i>Opportunities: flexibility, scalability, collaboration</i>	

Note: Our simulations are now based on LLaMA-3-405B, while the method remains applicable as the model scales up.

Decentralized Regime Training (Neurips 2022 Oral)

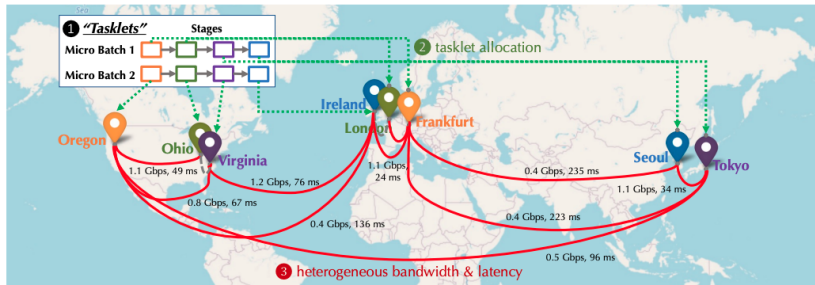


Figure: <https://openreview.net/pdf?id=UHoGOaGjEq>

Decentralized Regime Training

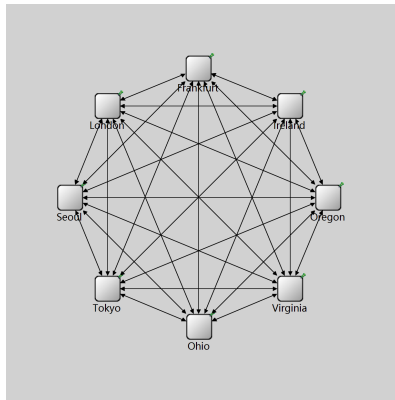


Figure: Their Topology
(By OMNeT++)

- Several computation centers might be located in a close area. They might form a clique, with one node dominating.
 - We propose a new topology to better handle this scenario.
- They applied data parallelism and pipeline parallelism
 - We added the third: tensor parallelism.
- We consider heterogeneous computation resources across clusters.

Outline

- 1 Background and Introduction
- 2 Proposed Method
- 3 Training Algorithm Implemented
- 4 Heterogeneous Training Time Analysis
- 5 Evaluation

Regional Subserver

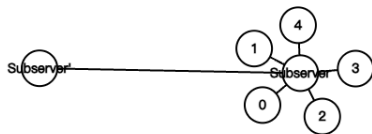


Figure: Introducing Subserver

- The edges between subserver are directed; they form a chain.
 - This allows us to split the model across subserver.
- Cluster-Subserver Connection: Large bandwidth and small latency
 - The subserver (with large bandwidth and low average latency) plays the role of a transit hub for ALLREDUCE.
- Inter-Subserver Connection: LARGE bandwidth to support LARGE batch parallel transmitting.

Topology Comparison in OMNeT++

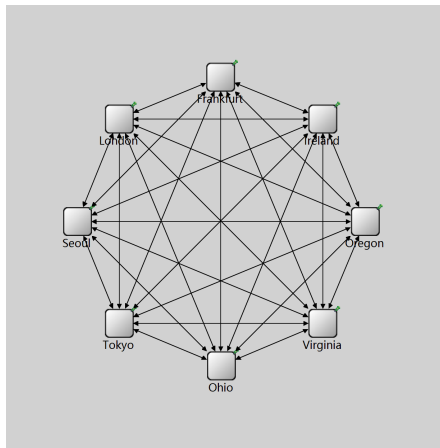


Figure: Their Topology

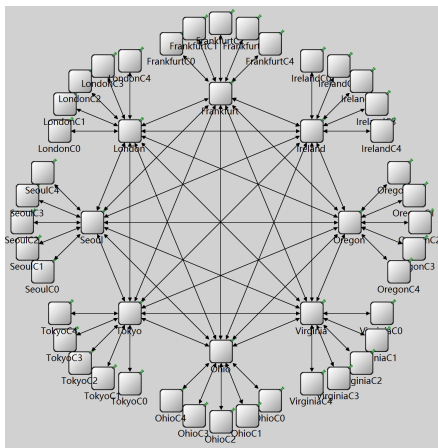


Figure: Our Topology

Recap: Parallelism

- **Data Parallelism:** The model is replicated across multiple devices, and each device processes a different mini-batch of input data.
- **Pipeline Parallelism:** The model is split into multiple layers, each assigned to a different device.
- **Tensor Parallelism (What they haven't done):** Individual model layers are partitioned across devices at the tensor level. Each device computes a slice of the tensor.

A Simple Transformer

A simple Transformer layer:

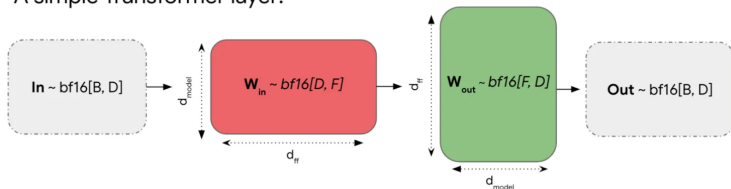


Figure:

<https://jax-ml.github.io/scaling-book/training/mixed-fsdp-and-tensor-parallelism>

Tensor Parallelism

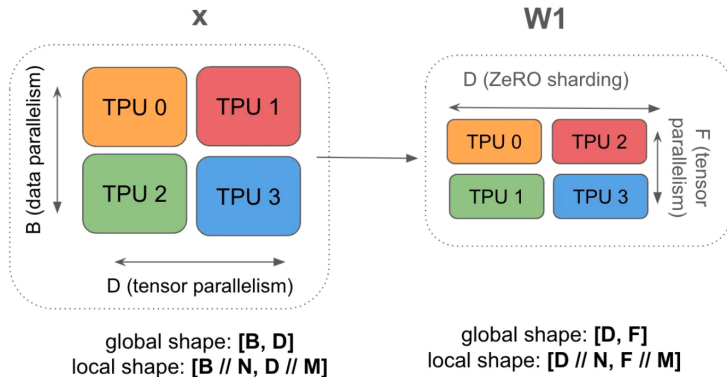


Figure:

<https://jax-ml.github.io/scaling-book/training/mixed-fsdp-and-tensor-parallelism>

Outline

- 1 Background and Introduction
- 2 Proposed Method
- 3 Training Algorithm Implemented**
- 4 Heterogeneous Training Time Analysis
- 5 Evaluation

Pipeline + Data Parallelism

Consider having two microbatches in a single batch, and three clusters:

Cluster \ Time	0	1	2	3	4	5	6
0	0F	1F			0B	1B	ReduceAll
1		0F	1F	0B	1B	ReduceAll	
2			0F	1F	ReduceAll		

Data Parallel: Forward Pass

Forward Pass (compute $\text{Loss}[B \times 1]$):

$\text{Tmp}[B \times F] = \text{In}[B \times D] \cdot \text{Win}[D \times F]$ // matrix multiplication along D

$\text{Out}[B \times D] = \text{Tmp}[B \times F] \cdot \text{Wout}[F \times D]$

$\text{Loss}[B] = \mathcal{L}(\text{Out}[B \times D])$ // element-wise loss computation

Data Parallel: Backward Pass

Backward Pass (compute gradients):

$$\begin{aligned}d\text{Out}[B \times D] &= \nabla \mathcal{L}(\text{Out}) \\d\text{Wout}[F \times D] \{UX\} &= \text{Tmp}[B \times F]^T \cdot_B d\text{Out}[B \times D] \\d\text{Wout}[F \times D] &= \text{AllReduce}(d\text{Wout} \{UX\}) \quad // \text{ async} \\d\text{Tmp}[B \times F] &= d\text{Out}[B \times D] \cdot_D \text{Wout}[F \times D]^T \\d\text{Win}[D \times F] \{UX\} &= \text{In}[B \times D]^T \cdot_B d\text{Tmp}[B \times F] \\d\text{Win}[D \times F] &= \text{AllReduce}(d\text{Win} \{UX\}) \quad // \text{ async} \\d\text{In}[B \times D] &= d\text{Tmp}[B \times F] \cdot_F \text{Win}[D \times F]^T\end{aligned}$$

Parallelism:

- Each worker processes a **different batch slice** (B dimension split).
- Gradients $d\text{Wout}$, $d\text{Win}$ computed locally then **AllReduced** (async).
- Computations marked with $\{UX\}$ indicate local per-worker contributions.

Outline

- 1 Background and Introduction
- 2 Proposed Method
- 3 Training Algorithm Implemented
- 4 Heterogeneous Training Time Analysis**
- 5 Evaluation

Notations and Parameters

Using Llama 3 405B.

Symbol	Meaning
$L = 126$	Number of layers in the model
$D = 16384$	d_{model} (hidden dimension)
$F = 53248$	d_{ff} (feed-forward dimension)
B	Microbatch size
n, m_i	n clusters, i -th cluster has m_i clients
i, j	The j -th client in i -th cluster
$b_{i,j}$	Per-device microbatch size
$(u), (d)$	Upload/download
$c_{i,j}$	Compute FLOPS
$v_{i,j}, w_{i,j}$	Intra-cluster link latency and bandwidth
$V_{i,j}, W_{i,j}$	Inter-cluster link latency and bandwidth

Time Cost Analysis I

For each microbatch, the per-cluster cost of forward and backward pass without gradient exchange is

$$T_{\text{layer},i} = 2 \max_{0 \leq j < m_i} \left\{ \frac{2b_{i,j}DFl_i}{c_{i,j}} + v_{i,j,(u)} + \frac{b_{i,j}Dl_i}{w_{i,j,(u)}} + v_{i,j,(d)} + \frac{b_{i,j}Dl_i}{w_{i,j,(d)}} \right\}, \quad (1)$$

which is the sum of the within-cluster compute and communication cost. Communication cost for single-layer gradient exchange (ReduceAll) is

$$T_{\text{ReduceAll},i} = 2 \max \left\{ \max_{1 \leq j < m_i} \left\{ v_{i,j,(u)} + \frac{DF}{w_{i,j,(u)}} \right\}, \max_{1 \leq j < m_i} \left\{ v_{i,j,(u)} + v_{i,j,(d)} + \frac{DF}{w_{i,j,(d)}} \right\} \right\}. \quad (2)$$

Time Cost Analysis II

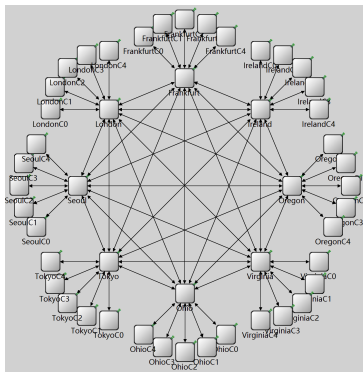
Let π_n be a permutation of $\{1, 2, \dots, n\}$, the inter-cluster communication cost is

$$\sum_{i=1}^{n-1} \left(v_{\pi_i, \pi_{i+1}} + \frac{BD}{W_{\pi_i, \pi_{i+1}}} \right). \quad (3)$$

Outline

- 1 Background and Introduction
- 2 Proposed Method
- 3 Training Algorithm Implemented
- 4 Heterogeneous Training Time Analysis
- 5 Evaluation

Abstracted Topology Using OMNeT++



- Simulation based on C++
- Considering latency, bandwidth, and computation resources

Result

Setup:

Parameter	Value
$\mathbb{N}$ (# input tokens)	10^9
$\mathbb{B}$ (batch size)	10^6
n (# clusters)	8
m_i (# clients in clusters)	5
$c_{i,j}$ (compute FLOPS)	$\approx 10^{14}$
$v_{i,j}, w_{i,j}$ (intra' latency, bandwidth)	(median consumer link in country)
$V_{i,j}, W_{i,j}$ (inter' latency, bandwidth)	(AWS instance benchmark)

Result:

B (microbatch size)	Time to train (days)
100,000	581.36
50,000	476.70
10,000	452.70

Thank you for listening!